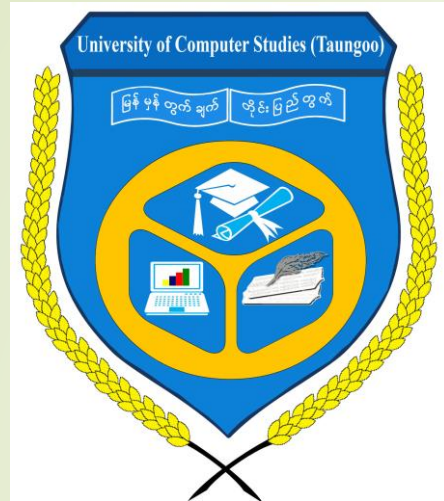




Introduction To Python

1

Dr. Hnin Lai Nyo

Faculty of Computer Science

UCS(Taungoo)

Contents

- Array
- Classes/Objects
- Modules

Arrays

- Python does **not have built-in** support for Arrays, but python lists can be used instead.
- To work with arrays in python, it will need to **import** a library, like **NumPy** library.

Example:

```
cars = ["Ford", "Volvo", "BMW"]
```

```
x = cars[0]      => Ford
```

```
car[0] = "Toyota"
```

```
x = len(cars)
```

```
for x in cars:
```

```
    print(x)
```

```
// Create an array
```

```
// Access the element of an array
```

```
// Modify the value of the first array item
```

```
// Length of an Array
```

```
// Looping Array Elements
```

Arrays

Example:

```
cars = ["Ford", "Volvo", "BMW"]  
for x in cars:  
    print(x)  
cars.append("Toyota")  
print(cars)  
print(cars.index("Volvo"))
```

Method	Description
append()	Adds an element at the end of the list
clear()	Removes all the elements from the list
copy()	Returns a copy of the list
count()	Returns the number of elements with the specified value
extend()	Add the elements of a list (end of the current list)
index()	Returns the index of the first element with the specified value
insert()	Adds an element at the specified position
pop()	Removes the element at the specified position
remove()	Removes the first item with the specified value
reverse()	Reverses the order of the list
sort()	Sorts the list

Classes/Objects

- Object : (its **properties** and **methods**)
- A class is like an object constructor, or a **blueprint** for creating objects.

- **Create a Class** (**class:**)

```
class ClassNm:
```

```
    x = 5
```

- **Create Object**

```
p1 = ClassNm( )
```

```
print(p1.x)
```

Classes/Objects

`__init__()` Function

- All classes have a function called `__init__()`, which is always executed when the class is being initiated.
- It is used to assign values to object properties, or other operations that are necessary to do when the object is being created.

Example:

```
class Person:
```

```
    def __init__(self, name, age):
```

```
        self.name = name
```

```
        self.age = age
```

```
p1 = Person("Mya Mya", 21)
```

```
print(p1.name)
```

=> Mya Mya

```
print(p1.age)
```

=> 21

Classes/Objects

`__str__()` Function

- This controls what should be returned when the class object is represented as string.
- If this function is not set, the string representation of an object is returned.

Example:

```
class Person:
```

```
    def __init__(self, name, age):
```

```
        self.name = name
```

```
        self.age = age
```

```
p1 = Person("Mya Mya", 21)
```

```
print(p1)
```

```
=> <__main__.Person object at 0x0000017010FDC590>
```


Classes/Objects

`__str__()` Function

Example:

```
class Person:
```

```
    def __init__(self, name, age):
```

```
        self.name = name
```

```
        self.age = age
```

```
    def __str__(self):
```

```
        return f "{self.name}, {self.age}"
```

```
p1 = Person("Mya Mya", 21)
```

```
print(p1)
```

```
=> Mya Mya, 21
```


Classes/Objects

Object Methods

- Objects can also contain methods(functions).

Example:

```
class Person:
```

```
    def __init__(self, name, age):
```

```
        self.name = name
```

```
        self.age = age
```

```
    def funname(self):
```

```
        print("Hi! Good Morning " + self.name)
```

```
p1 = Person("Mya Mya", 21)
```

```
print(p1.name)
```

```
=> Mya Mya
```

```
print(p1.age)
```

```
=> 21
```

```
p1.funname( )
```

```
=> Hi! Good Morning Mya Mya
```

Classes/Objects

The self Parameter

- The **self** parameter is a reference to the current instance of the class.
- It is used to access variables that belong to the class.
- It does not have to be named self and it has to be the first parameter of any function in the class.

Example:

```
class Person:
```

```
    def __init__(xyza, name, age):
```

```
        xyza.name = name
```

```
        xyza.age = age
```

```
    def funname(abc):
```

```
        print("Hi! Good Morning " + abc.name)
```

```
p1 = Person("Mya Mya", 21)
```

```
p1.funname()
```

=> Hi! Good Morning Mya Mya

Modules

11

- A file containing a set of functions you want to include in your application.

Create a Module

- To create a module just **save** the code you want in a file with the file extension **.py**

Example:

```
def greeting(name):  
    print("Hello, " + name)
```

mymodule.py

```
import mymodule  
mymodule.greeting("Richard")
```

Module_Name

Function_Name

```
person={"name": "John",  
        "age":36, "country" : "Italy" }
```

mymodule.py

```
import mymodule  
a = mymodule.person["age"]  
print(a)
```

Modules

Re-naming a Module

- You can create an alias when you import a module, by using the **as** keyword.

Example:

```
def greeting(name):
```

```
    print("Hello, " + name)
```

mymodule.py

```
import mymodule as mx
```

```
mx.greeting("Richard")
```

Module_Name

Function_Name

Modules

13

Built-in Modules

- You can create an alias when you import a module, by using the **as** keyword.

Example:

```
import platform
x=platform.system( )
print(x)
y=dir(platform)
print(y)
```

Modules

Import From Module

- To import only parts from a module, by using the **from** keyword.

Example:

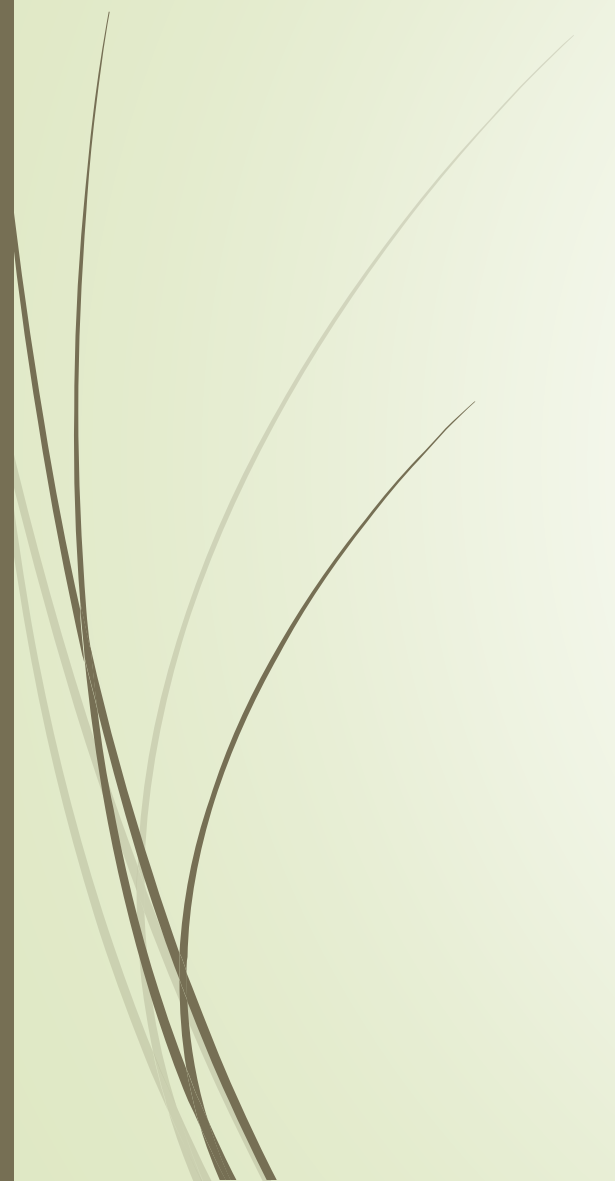
```
def greeting(name):  
    print("Hello, " + name)  
person={"name": "John",  
        "age":36, "country": "Italy"}
```

mymodule.py

```
from mymodule import person  
print(person["age"])
```

Exercises

- <https://pynative.com/python-operators-and-expression-quiz/>
- <https://pynative.com/python-variables-and-data-types-quiz/>
- <https://pynative.com/basic-python-quiz-for-beginners/>
- <https://pynative.com/python/quizzes/>
- <https://pynative.com/python-basic-exercise-for-beginners/>
- https://www.w3schools.com/python/python_functions.asp



Thank You